Envelope для возврата результатов.

Содержание

1. Единый формат результата запроса 2
2. Учим `Envelope` сериализоваться в JSON 3
3. Учим `Envelope` преобразовываться в `Response` 5
   - 3. 1. Статус коды 5
   - 3. 2. Преобразование в ответ 5
   - 3. 3. Функции для отдельных статус-кодов 6
4. Учим ошибки преобразовываться в `String` 7
   - 4. 1. Реализация `Display` 8
   - 4. 2. Использование `thiserror` 8
   - 4. 3. Адаптер ошибок базы данных 9

Задачи 11

1. Единый формат результата запроса

В `Ohkami` фреймворке используется `Response` для того, чтобы вернуть результаты запроса.

```
pub async fn add_item(
    Json(payload): Json<AddItemPayload>,
    Context(context): Context<'_, ItemsHttpContext>,
) -> Response
```

В обработке запросов могут быть сценарии, когда нам нужно вернуть результаты запроса, например в виде успешного результата с телом ответа, или с ошибкой:

```
let name: Name = Name::new_with_format_and_length_check(
    payload.name,
    vec![FormatOptions::Trim, FormatOptions::FirstLetterToUpper],
    MIN_ITEM_NAME_LENGTH,
    MAX_ITEM_NAME_LENGTH,
)?;
```

или

```
let connection = context.get_connection().await
    .map_err(|error| )?; // а во что превращать error ошибку?
```

Здесь каждая функция: `new_with_format_and_length_check` и `get_connection` могут вернуть ошибки, которые нужно обработать.

Но проблема в том, что это разные типы ошибок, которые нужно преобразовать в единый тип для возврата в `Response`.

Значит нам нужен механизм, который будет преобразовывать разные типы ошибок в единый тип для возврата в `Response`.

Идея — создать `Envelope` для возврата результатов.

Информация:

Что такое `Envelope`? `Envelope` — это оболочка для возврата результатов запроса. Она может содержать:

- Данные
- Ошибки
- Статус
- Дату операции

Используя эту информацию, создадим структуру `Envelope` для возврата результатов.

```
pub struct Envelope<T>
where
    T: Serialize,
{
    pub data: Option<T>,
    pub error: Option<String>,
    pub status: i32,
```

```
    pub date: UtcDateTime,
}
```

Хорошо, теперь мы можем использовать `Envelope` для возврата результатов запросов.

```
let name: Name = match Name::new_with_format_and_length_check(
    payload.name,
    vec![FormatOptions::Trim, FormatOptions::FirstLetterToUpper],
    MIN_ITEM_NAME_LENGTH,
    MAX_ITEM_NAME_LENGTH,
) {
    Err(error) => {
        // При ошибке можем вернуть Envelope с ошибкой
        let envelope = Envelope::<Uuid> {
            data: None,
            error: Some(error.to_string()),
            status: 400,
            date: UtcDateTime::now(),
        };
        return Response::new(Status::BadRequest).with_json(envelope);
    },
    Ok(name) => name
};
```

Есть два препятствующих момента:

1. `Envelope` должен уметь сериализоваться в JSON.
2. `Envelope` должен уметь преобразовываться в `Response`.
3. Ошибки должны уметь преобразовываться в `String`.

Будем решать эти проблемы поэтапно.

2. Учим `Envelope` сериализоваться в JSON

Чтобы `Envelope` мог сериализоваться в JSON, нам нужно добавить `Serialize` и `Deserialize` для `Envelope`.

По-умолчанию в Rust нет встроенной поддержки для сериализации и десериализации структур данных. Для этого нам нужно использовать внешние библиотеки, такие как `serde`.

Подключим библиотеку `serde` и `serde_json`:

В `Cargo.toml` убедитесь, что у вас есть следующее:

```
[dependencies.serde]
version = "1.0"
features = ["derive"]

[dependencies.serde_json]
version = "1.0"
```

Импортируйте следующие конструкции:

```
use serde::{Deserialize, Serialize};
use serde_json;
```

Добавьте `#[derive(Serialize)]` для `Envelope`:

```
#[derive(Serialize)]
struct Envelope<T>
where
    T: Serialize,
{
    data: Option<T>,
    error: Option<String>,
    status: u16,
    date: UtcDateTime,
}
```

После добавления `Serialize` у нас может быть ошибка, связанная с тем, что `UtcDateTime` не поддерживает сериализацию.

```
the trait bound `UtcDateTime: serde::Serialize` is not satisfied
```

Для решения этой проблемы нам нужно научить `UtcDateTime` сериализоваться в JSON.

Добавьте функцию, которая будет преобразовывать `UtcDateTime` в строку, которую можно сериализовать в JSON:

```
fn serialize_utc_datetime<S>(
    value: &UtcDateTime,
    serializer: S,
) -> Result<S::Ok, S::Error>
where
    S: serde::Serializer,
{
    let text = value
        .format(&time::format_description::well_known::Rfc3339)
        .map_err(serde::ser::Error::custom)?;
    serializer.serialize_str(&text)
}
```

Дальше, в `Envelope` добавьте следующий `template` (автогенерируемый код) над полем даты:

```
#[derive(Serialize)]
pub struct Envelope<T>
where
    T: Serialize,
{
    pub data: Option<T>,
    pub error: Option<String>,
    pub status: i32,
    #[serde(serialize_with = "serialize_utc_datetime")]
    pub date: UtcDateTime,
}
```

Теперь ошибка должна исчезнуть.

3. Учим `Envelope` преобразовываться в `Response`

`Response` — это структура, которая содержит информацию о результате запроса, включая статус, заголовки и тело ответа.

`Response` должен знать, когда ему быть ошибкой, а когда быть успешным. Мы можем делать это суждение на основе статус-кода.

3. 1. Статус коды

Статус коды можно распределить по следующим категориям:

- 2xx: Успешные запросы
- 4xx: Ошибки клиента
- 5xx: Ошибки сервера

К статус кодам 4xx относятся следующие:

- 400: Ошибка в запросе клиента (`Bad Request`)
- 401: Неавторизованный запрос (`Unauthorized`)
- 403: Запрещенный запрос (`Forbidden`)
- 404: Не найдено (`Not Found`)
- 405: Метод не поддерживается (`Method Not Allowed`)
- 409: Конфликт (`Conflict`)
- 422: Недопустимые данные (`Unprocessable Entity`)

К статус кодам 5xx относятся следующие:

- 500: Внутренняя ошибка сервера (`Internal Server Error`)
- 501: Не поддерживаемый метод (`Not Implemented`)
- 503: Сервис недоступен (`Service Unavailable`)

К статус кодам 2xx относятся следующие:

- 200: Успешный запрос (`OK`)
- 201: Создан новый ресурс (`Created`)
- 204: Ресурс успешно удален (`No Content`)

3. 2. Преобразование в ответ

Для того, чтобы удобно создавать `Response` на основе `Envelope` определим следующую функцию:

```
impl<T> Envelope<T>
where
    T: Serialize,
{
    pub fn into_response(self) -> Response {
        let status = self.resolve_ohkami_status_code();
        let body = serde_json::to_string(&self).unwrap();
        Response::new(status).with_text(body)
    }
}
```

В этой функции нам нужно сопоставить `i32` поле `status` с соответствующим статус-кодом из `Ohkami` :

Для сопоставления статус-кода из `i32` в статус-код `Ohkami` мы можем использовать следующий код:

```
impl<T> Envelope<T>
where
    T: Serialize,
{
    fn resolve_ohkami_status_code(&self) -> Status {
        match self.status {
            100..=103
            | 200..=208
            | 226
            | 300..=304
            | 307..=308
            | 400..=401
            | 403..=418
            | 421..=424
            | 426
            | 428..=429
            | 431
            | 451
            | 500..=508
            | 510..=511 => Status::from(self.status as u16),
            _ => Status::InternalServerError,
        }
    }
}
```

Здесь, данный код просто берёт диапазон статус-кодов и возвращает соответствующий статус-код `Ohkami`. В случае, если статус-код не соответствует ни одному из диапазонов, возвращается `Status::InternalServerError`.

3. 3. Функции для отдельных статус-кодов

Для того, чтобы удобно создавать `Response` на основе `Envelope` определим следующие функции под каждый статус-код:

```
// хелпер для создания Envelope с ошибкой
fn current_error(error: Option<String>) -> Self {
    Envelope {
        data: None,
        error,
        status: 400,
        date: UtcDateTime::now(),
    }
}

// хелпер для создания Envelope с успешным результатом
fn current_success(data: Option<T>) -> Self {
    Envelope {
        data,
        error: None,
        status: 200,
```

```
        date: UtcDateTime::now(),
    }
}

// 404 статус-код
pub fn bad_request(error: Option<String>) -> Response {
    let envelope = Self::current_error(error);
    return Response::new(Status::BadRequest).with_json(envelope);
}

// 200 статус-код
pub fn ok(data: Option<T>) -> Response {
    let envelope = Self::current_success(data);
    return Response::new(Status::OK).with_json(envelope);
}

// 500 статус-код
pub fn internal_error(error: Option<String>) -> Response {
    let envelope = Self::current_error(error);
    return Response::new(Status::InternalServerError).with_json(envelope);
}

// 409 статус-код
pub fn conflict(error: Option<String>) -> Response {
    let envelope = Self::current_error(error);
    return Response::new(Status::Conflict).with_json(envelope);
}
```

4. Учим ошибки преобразовываться в `String`.

Сейчас последним препятствием является то, что ошибки не преобразуются в `String` автоматически.

```
let name: Name = match Name::new_with_format_and_length_check(
    payload.name,
    vec![FormatOptions::Trim, FormatOptions::FirstLetterToUpper],
    MIN_ITEM_NAME_LENGTH,
    MAX_ITEM_NAME_LENGTH,
) {
    Err(error) => {
        return Envelope::bad_request(Some(error.to_string()));
//                                       ^^^^^^^^^^^^^^^^^^^^
//                                       НЕЛЬЗЯ
    }
    Ok(name) => name,
};
```

Посмотрим, что за ошибку мы получили:

```
pub enum NameError {
    NameIsEmpty,
    NameTooLong,
```

```
    NameExceedsMaxLength,
    NameLessThanMinLength,
}
```

4. 1. Реализация `Display`

Для того, чтобы ошибки преобразовывались в `String`, нужно реализовать для них трейт `std::fmt::Display`:

```
impl std::fmt::Display for NameError {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        match self {
            NameError::NameIsEmpty => write!(f, "Name is empty"),
            NameError::NameTooLong => write!(f, "Name is too long"),
            NameError::NameExceedsMaxLength =>
                write!(f, "Name exceeds max length"),
            NameError::NameLessThanMinLength =>
                write!(f, "Name less than min length"),
        }
    }
}
```

4. 2. Использование `thiserror`

Куда более проще — это подключить библиотеку `thiserror` и использовать её для реализации `Display`. Она автоматически за вас будет генерировать `Display` и `Error` для ошибок при помощи атрибутов `#[derive]`.

Подключим библиотеку в `Cargo.toml`:

```
[dependencies.thiserror]
version = "2"
```

Теперь мы можем удалить `Display` для `NameError` и `ItemError` и заменить его на `thiserror`:

```
#[derive(Debug, thiserror::Error)]
pub enum NameError {
    #[error("validation.name_empty")]
    NameIsEmpty,
    #[error("validation.name_too_long")]
    NameTooLong,
    #[error("validation.name_exceeds_max_length")]
    NameExceedsMaxLength,
    #[error("validation.name_less_than_min_length")]
    NameLessThanMinLength,
}
```

Теперь, следующий код будет работать:

```
let name: Name = match Name::new_with_format_and_length_check(
    payload.name,
    vec![FormatOptions::Trim, FormatOptions::FirstLetterToUpper],
```

```
    MIN_ITEM_NAME_LENGTH,
    MAX_ITEM_NAME_LENGTH,
) {
    Err(error) => {
        return Envelope::bad_request(Some(error.to_string()));
//                                       ^^^^^^^^^^^^^^^^^^^
//                                       МОЖНО
    }
    Ok(name) => name,
};
```

4. 3. Адаптер ошибок базы данных

Примечание:

Важно понимать. Большинство библиотек, которые вы подключаете, автоматически `thiserror` могут не содержать. Например ошибки с базой данных.

В таком случае, вам нужно создавать поверх ошибок библиотек свои адаптеры, чтобы они реализовали `Display` и `Error`. Например, для ошибок с базой данных можно создать адаптер `DatabaseError`:

```
#[derive(Debug, thiserror::Error)]
pub enum DatabaseError {
    // Ошибка подключения к базе данных
    #[error("database.connection_error")]
    ConnectionError,

    // Ошибка при отсутствии изменений в записи
    #[error("database.no_changed_entries_error")]
    NoChangedEntriesError,

    // Ошибка при отображении записи
    #[error("database.entry_mapping_error")]
    EntryMappingError,
}
```

И использовать эти ошибки в ваших функциях, например в других трейтах-адаптерах.

```
// алиас, чтобы не писать длинный:
//    impl Future<Output = Result<(), DatabaseError>>
// а писать короче:
//    impl Future<Output = StorageResult<()>>;
type StorageResult<T> = Result<T, DatabaseError>;
//                              ^^^^^^^^^^^^^
//                              адаптируемая ошибка.

pub trait ItemsStorage {
    fn add(&self, item: Item) -> impl Future<Output = StorageResult<()>>;
    //                                                  ^^^^^^^^^^^^^^^^^^^^^
    //                                                  использование
```

```
    fn remove(&self, item: Item) -> impl Future<Output = StorageResult<()>>;
    //                                                  ^^^^^^^^^^^^^^^^^^^^^
    //                                                  использование

    fn get(&self, specification: ItemSpecification) ->
StorageResult<Option<Item>>;
    //                                                  ^^^^^^^^^^^^^^^^^^^^^
    //                                                  использование

    fn update(&self, item: &Item) -> impl Future<Output = StorageResult<()>>;
    //                                                  ^^^^^^^^^^^^^^^^^^^^^
    //                                                  использование
}
```

Задачи

Задача 1. Создать оболочку результата

Создайте `Envelope` с реализацией как в примере.

Задача 2. Унифицировать ошибки адаптеров

Сделайте так, чтобы трейты-адаптеры работы с `SQLite` возвращали `StorageResult<T>`, где `StorageResult<T>` это `Result<T, DatabaseError>`. Для этого нужно определить `DatabaseError` как в примере, а так же `type-alias` `type X = Y`.

Задача 3. Подключить обработку перечислений ошибок

Найдите все ваши перечисления ошибок (например `NameError`). И подключите в них `#[derive(Debug, thiserror::Error)]` как в примере.

Задача 4. Исправить ошибки компиляции

Исправьте возможные ошибки компиляции которые могут возникнуть при изменении кода.